Chapter 3: Python Programming Essentials

This chapter reviews Python essentials for data science, helping you write cleaner, faster, and more efficient code.

1. Variables

Variables store data that can be used and modified in a program. Python automatically **detects the type.**

Type Conversion in Python

Type conversion (or type casting) is the process of converting a variable from one data type to another. Python provides built-in functions for this.

In [1]:
```
# Examples of variables
name = "Ali"                  # string
age = 25                      # integer
height = 16.1                  # float
is_student = True            # boolean
```

Data Types

Type	Example	Description
int	42	Integer number
float	9.81	Decimal number
str	"Python"	String / text
bool	True, False	Boolean value
list	["apple", "banana"]	Ordered collection
dict	{"city": "Paris"}	Key-value pairs
tuple	(1, 2, 3)	Immutable ordered collection
set	{1, 2, 3}	Unordered collection of unique items

In [2]:
```
# Type Conversion in Python
x = "100"
y = int(x)        # string -> int
z = float(x)      # int -> float
a = str(z)        # float -> string
b = bool(a)       # string -> bool (non-empty -> True)

print(x, y, z, a, b)
print(type(y), type(z), type(a), type(b))
```

```
100 100 100.0 100.0 True
<class 'int'> <class 'float'> <class 'str'> <class 'bool'>
```

2. Operators

Python supports arithmetic, comparison, and logical operators.

- **Arithmetic:** `+`, `-`, `*`, `/`, `//`, `%`, `**`
- **Comparison:** ==, !=, >, <, >=, <=
- **Logical**: and, or, not

In [3]:
```
# Arithmetic: `+`, `-`, `*`, `/`, `//`, `%`, `**`

a, b = 10, 3
print(a + b)
print(a ** b)
print(a // b)
print(a / b)
```

```
13
1000
3
3.3333333333333335
```

In [4]:
```
# Comparison: ==, !=, >, <, >=, <=
print(a > b)   # True
print(a == b)  # False
```

```
True
False
```

In [5]:
```
# Logical: and, or, not

x, y = True, False
print(x and y) # False
print(not x)   # False
```

```
False
False
```

3. Control Flow: Conditionals and Loops

Control flow allows to dictate the order in which statements are executed. Python supports **conditionals** and **loops.**

In [6]:
```
# Conditionals (`if`, `elif`, `else`): Used to execute code based on a condition.

grade = 13

if grade > 12:
    print("good")
elif grade >= 10:
    print("accept")
else:
    print("fail")
```

```
good
```

In [7]:
```
# Loops
# for loop - iterates over a sequence (list, string, range, etc.)
fruits = ["apple", "banana", "cherry"]
```

```
for fruit in fruits:
    print(fruit)

# for loop over a range of numbers
for i in range(3):
    print(i)
```

```
apple
banana
cherry
0
1
2
```

In [8]:
```
# while loop - repeats as long as a condition is True
count = 0
while count < 5:
    print(count)
    count += 1
```

```
0
1
2
3
4
```

Loop Control Statements

- **break** – exit the loop immediately
- **continue** – skip the rest of the current iteration
- **pass** – does nothing, acts as a placeholder

In [9]:
```
for i in range(5):
    if i == 3:
        break    # exit loop
    print(i)

for i in range(5):
    if i == 2:
        continue # skip this iteration
    print(i)

for i in range(5):
    if i == 1:
        pass     # placeholder, does nothing
    print(i)
```

```
0
1
2
0
1
3
4
0
1
2
3
4
```

4. Functions in Python

Functions in Python let you group code into reusable blocks, making your programs cleaner, more organized, and easier to maintain.

In [10]:
```
# Defining a Function

def greet(name):
    return f"Hello, {name}!"
# call of a function
print(greet("Ali"))
```

```
Hello, Ali!
```

In [11]:
```
# Parameters and Return Values
def add(x, y):
    return x + y
```

In [12]:
```
# You can return multiple values:
def split_name(full_name):
    parts = full_name.split(" ")
    first = parts[0]
    last = parts[-1]
    return first, last

fname, lname = split_name("Ali BENALI")
print(fname, lname)
```

```
Ali BENALI
```

Lambda Functions in Python

A lambda function is a small, anonymous function (no name, defined in one line).

General syntax: lambda arguments: expression

- lambda → keyword
- arguments → input(s)
- expression → returned result (automatically returned, no return keyword needed)

In [13]:
```
# Basic Example
square = lambda x: x**2 # This is equivalent to: def square(x):     return x**2

print(square(5))
```

```
25
```

In [14]:
```
# With Multiple Arguments
add = lambda a, b: a + b
print(add(3, 7))
```

```
10
```

In [15]:
```
# Used Inside map()
# map() applies a function to each element of an iterable.

numbers = [1, 2, 3, 4]
```

```
squares = list(map(lambda x: x**2, numbers))
print(squares)
```

```
[1, 4, 9, 16]
```

In [16]:
```
# Used Inside filter()

# filter() keeps only the elements that satisfy a condition.

numbers = [5, 10, 15, 20, 25]
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens)
```

```
[10, 20]
```

In [17]:
```
# With sorted()
pairs = [(1, 'one'), (3, 'three'), (2, 'two')]
sorted_pairs = sorted(pairs, key=lambda x: x[1])
print(sorted_pairs)
```

```
[(1, 'one'), (3, 'three'), (2, 'two')]
```

In [18]:
```
# In Pandas .apply()
import pandas as pd
data = pd.Series([1, 2, 3, 4])
squared = data.apply(lambda x: x**2)
print(squared)
```

```
0     1
1     4
2     9
3    16
dtype: int64
```

5. Essential Python Data Structures

Python provides powerful built-in data structures that are critical for data analysis, machine learning, and general programming. They let you store, access, and manipulate information efficiently.

5.1. Lists

Definition: An ordered, mutable collection (can be changed after creation).

Use cases: Storing sequences of items such as records, rows of data, or intermediate results.

In [19]:
```
fruits = ["apple", "banana", "cherry"]

print(len(fruits))      # Length → 3
```

```
3
```

In [20]:
```
# Main List Operations in Python

# 1. Create a list
fruits = ["apple", "banana", "cherry"]
print("Original list:", fruits)
```

```
Original list: ['apple', 'banana', 'cherry']
```

In [21]:
```
# 2. Accessing elements
print("First element:", fruits[0])
print("Last element:", fruits[-1])
```

```
# 3. Adding elements
fruits.append("kiwi")        # Add at the end
fruits.insert(1, "orange")   # Insert at index 1
print("After adding:", fruits)
```

```
First element: apple
Last element: cherry
After adding: ['apple', 'orange', 'banana', 'cherry', 'kiwi']
```

In [22]:
```
# 4. Removing elements
fruits.remove("banana")   # Remove the element by its value
fruits.pop(0)             # Remove the element at index 0 (the first element)
del fruits[1]             # Delete the element at index 1
print("After removals:", fruits)

# 5. Modifying elements
fruits[0] = "mango"       # Replace the first element with "mango"
print("After modifying:", fruits)
```

```
After removals: ['orange', 'kiwi']
After modifying: ['mango', 'kiwi']
```

In [23]:
```
# 6. Length and search
print("Length:", len(fruits))
print("Is 'kiwi' in list?", "kiwi" in fruits)
```

```
Length: 2
Is 'kiwi' in list? True
```

In [24]:
```
# 7. Concatenation and repetition
tropical = ["pineapple", "papaya"]
all_fruits = fruits + tropical
print("Concatenated:", all_fruits)
print("Repeated:", all_fruits * 3)
```

```
Concatenated: ['mango', 'kiwi', 'pineapple', 'papaya']
Repeated: ['mango', 'kiwi', 'pineapple', 'papaya', 'mango', 'kiwi', 'pineapple', 'papaya', 'ma
ngo', 'kiwi', 'pineapple', 'papaya']
```

In [25]:
```
# 8. Iteration
print("Iterating over list:")
for fruit in all_fruits:
    print("-", fruit)
```

```
Iterating over list:
- mango
- kiwi
- pineapple
- papaya
```

In [26]:
```
# 9. Sorting and reversing
numbers = [3, 1, 4, 1, 5]
numbers.sort()
print("Sorted ascending:", numbers)
numbers.sort(reverse=True)
print("Sorted descending:", numbers)
numbers.reverse()
print("Reversed order:", numbers)
```

```
Sorted ascending: [1, 1, 3, 4, 5]
Sorted descending: [5, 4, 3, 1, 1]
Reversed order: [1, 1, 3, 4, 5]
```

In [27]:
```
# 10. Slicing
letters = ["a", "b", "c", "d", "e"]
print("Slice 1:4:", letters[1:4])
print("First 3:", letters[:3])
print("Last 3:", letters[3:])
```

```
print("all:", letters[:])
print("Every 2nd:", letters[::2])
```

```
Slice 1:4: ['b', 'c', 'd']
First 3: ['a', 'b', 'c']
Last 3: ['d', 'e']
all: ['a', 'b', 'c', 'd', 'e']
Every 2nd: ['a', 'c', 'e']
```

5.2. Tuples

A tuple is like a list, but immutable (cannot be changed after creation).

In [28]:
```
dimensions = (1920, 1080)
```

5.3. Dictionaries in Python

A dictionary stores data as key-value pairs.

Keys must be unique and immutable (string, number, tuple), values can be any type.

In [29]:
```
# Creating a dictionary
person = {
    "name": "Ali",
    "age": 30,
    "city": "Alger"
}

# Common operations
print("Original dictionary:", person)

# Access value by key
print("Name:", person["name"])
```

```
Original dictionary: {'name': 'Ali', 'age': 30, 'city': 'Alger'}
Name: Ali
```

In [30]:
```
# Add or update a key-value pair
person["email"] = "alice@example.com"
person["age"] = 31
print("After adding email and updating age:", person)
```

```
After adding email and updating age: {'name': 'Ali', 'age': 31, 'city': 'Alger', 'email': 'ali
ce@example.com'}
```

In [31]:
```
# Remove a key-value pair
del person["city"]
print("After removing city:", person)
```

```
After removing city: {'name': 'Ali', 'age': 31, 'email': 'alice@example.com'}
```

In [32]:
```
# Get all keys, values, and items
print("Keys:", person.keys())
print("Values:", person.values())
print("Items (key-value pairs):", person.items())
```

```
Keys: dict_keys(['name', 'age', 'email'])
Values: dict_values(['Ali', 31, 'alice@example.com'])
Items (key-value pairs): dict_items([('name', 'Ali'), ('age', 31), ('email', 'alice@example.co
m')])
```

In [33]:
```
# Check if a key exists
print("Is 'name' a key?", "name" in person)
print("Is 'city' a key?", "city" in person)
```

```
Is 'name' a key? True
Is 'city' a key? False
```

In [34]:
```
# Safe access with get (avoids error if key is missing)
print("Get email safely:", person.get("email"))
print("Get city safely:", person.get("city", "Not found"))
```

```
Get email safely: alice@example.com
Get city safely: Not found
```

5.4. Sets

A set is an unordered collection of unique elements. Useful for:

-Removing duplicates, membership tests, and set operations.

-Set operations: union, intersection, difference

In [35]:
```
unique_tags = set(["data", "science", "data", "python"])
print(unique_tags) # {'python', 'data', 'science'}
```

```
{'data', 'science', 'python'}
```

In [36]:
```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
print (set1 & set2)
print(set1 | set2)
print(set1 - set2)
print(set1 ^ set2)
```

```
{3}
{1, 2, 3, 4, 5}
{1, 2}
{1, 2, 4, 5}
```

In [37]:
```
# Add an element
set1.add(6)
print("After adding:", set1)

# Remove an element (KeyError if not found)
set1.remove(6)
print("After removing:", set1)
```

```
After adding: {1, 2, 3, 6}
After removing: {1, 2, 3}
```

In [38]:
```
set1.discard(10)
print("After discarding 10 (not in set):", set1)

# Pop an arbitrary element
removed = set1.pop()
print("Removed element with pop():", removed)
print("After pop:", set1)

# Check membership
print("Is 2 in the set?", 2 in set1)
print("Is 5 in the set?", 5 in set1)
```

```
After discarding 10 (not in set): {1, 2, 3}
Removed element with pop(): 1
After pop: {2, 3}
Is 2 in the set? True
Is 5 in the set? False
```

5.5. Strings: Manipulating Text Data

In [39]:
```python
## String Basics

text = "Data Science"

# Convert to lowercase
text.lower()  # Output: "data science"

# Convert to uppercase
text.upper()  # Output: "DATA SCIENCE"

# Replace a substring
text.replace("Data", "AI")  # Output: "AI Science
```

Out[39]: 'AI Science'

In [40]:
```python
#String Indexing and Slicing
#Python strings are sequences, so you can access individual characters or substrings:
text = "Data Science"

# Indexing
print(text[0]  )

# Slicing (first 4 characters)
print(text[:4] )
# Negative indexing (last character)
print(text[-1] )
```

```
D
Data
e
```

f-Strings (String Interpolation)

f-Strings provide a modern and readable way to embed variables inside strings:

In [41]:
```python
name = "Ali"
age = 30

# Embed variables in a string
f"My name is {name} and I am {age} years old."
```

Out[41]: 'My name is Ali and I am 30 years old.'

In [42]:
```python
# Expressions can also be embedded
f"Next year, I will be {age + 1} years old."
```

Out[42]: 'Next year, I will be 31 years old.'

Useful Methods

Method	Purpose	Example
`.split()`	Break string into list	`"Data Science".split()` → `['Data', 'Science']`
`.join()`	Combine list into string	`" ".join(['Python', 'is', 'fun'])` → `"Python is fun"`
`.strip()`	Remove leading/trailing whitespace	`"  Data  ".strip()` → `"Data"`
`.startswith()`	Check start pattern	`"Data Science".startswith("Data")` → `True`
`.find()`	Locate substring	`"Data Science".find("Science")` → `5`

5.6. Stacks and Queues in Python

Stack (LIFO - Last In, First Out)

A **stack** is a data structure where the **last element added** is the **first one removed.**
In Python, a list can be used as a stack.

In [43]:
```
stack = []
stack.append('A')   # Push
stack.append('B')
stack.append('C')
print("Stack:", stack)

stack.pop()         # Pop (removes last item)
print("After pop:", stack)
```

```
Stack: ['A', 'B', 'C']
After pop: ['A', 'B']
```

Queue (FIFO - First In, First Out)

A queue is a data structure where the first element added is the first one removed. Python's collections.deque is perfect for this.

collections is a **built-in Python module** that provides **specialized data structures** — more powerful and efficient than basic types like `list`, `dict`, or `tuple`

In [44]:
```
from collections import deque

queue = deque()
queue.append('A')    # Enqueue
queue.append('B')
queue.append('C')
print("Queue:", queue)

queue.popleft()      # Dequeue (removes first item)
print("After popleft:", queue)
```

```
Queue: deque(['A', 'B', 'C'])
After popleft: deque(['B', 'C'])
```

6. List Comprehensions in Python

List comprehensions provide a concise and expressive way to create, filter, and transform lists in a single line of code. They often replace traditional for loops when building a new list from an existing sequence (like a list, tuple, or range()).

Use cases:

- Data preprocessing (filtering and transforming lists)
- Feature engineering (creating combinations, transformations)
- Concise creation of dictionaries or sets
- Flattening nested structures
- **General syntax:**

[new_element for element in iterable if condition]

In [45]:
```
# Basic example
# Create a list of squares
squares = [x**2 for x in range(5)]
print(squares)
```

```
[0, 1, 4, 9, 16]
```

In [46]:
```
# With a condition (filtering)
even_squares = [x**2 for x in range(5) if x % 2 == 0]
print(even_squares)
```

```
[0, 4, 16]
```

In [47]:
```
# With nested loops
# Cartesian product
pairs = [(x, y) for x in [1, 2] for y in [3, 4]]
print(pairs)
```

```
[(1, 3), (1, 4), (2, 3), (2, 4)]
```

In [48]:
```
# Nested list comprehension (e.g., flattening a 2D list)
matrix = [[1, 2, 3], [4, 5, 6]]
flattened = [num for row in matrix for num in row]
print(flattened)
```

```
[1, 2, 3, 4, 5, 6]
```

In [49]:
```
# Conditional transformation (e.g., replace odd numbers with 'odd')
transformed = [x if x % 2 == 0 else 'odd' for x in range(6)]
print(transformed)
```

```
[0, 'odd', 2, 'odd', 4, 'odd']
```

In [50]:
```
# Dictionary comprehension example
squares_dict = {x: x**2 for x in range(5)}
print(squares_dict)
```

```
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

In [51]:
```
# Set comprehension example
unique_squares = {x**2 for x in [1, 2, 2, 3, 3, 3]}
print(unique_squares)
```

```
{1, 4, 9}
```

7. Reading and Writing Files in Python

Python provides simple ways to read and write **text files**, **CSV files**, and **JSON files**.

7.1 Text Files

Reading a text file

In [52]:
```
# Open file in read mode
with open("example.txt", "r") as file:
    content = file.read()
    print(content)

# reading line by line:
with open("example.txt", "r") as file:
    for line in file:
        print(line.strip())  # removes newline characters


# Reading All Lines into a List
with open("example.txt", "r") as file:
    lines = file.readlines()
    print(lines)
```

```
Hello, world!
Python file handling is easy.
Adding a new line.
Hello, world!
Python file handling is easy.
Adding a new line.
['Hello, world!\n', 'Python file handling is easy.\n', 'Adding a new line.']
```

Writing to a text file

In [53]:
```
## Open file in write mode
with open("example.txt", "w") as file:
    file.write("Hello, world!\n")
    file.write("Python file handling is easy.")
## Appending to a text file
with open("example.txt", "a") as file:
    file.write("\nAdding a new line.")
```

7.2. CSV Files

Python has a built-in csv module for reading and writing CSV files.

In [54]:
```
## Reading a CSV file
import csv
with open("data.csv", "r") as csvfile:
    reader = csv.reader(csvfile)
    for row in reader:
        print(row)
```

```
['Name', 'Age', 'City']
['Alice', '25', 'Paris']
['Bob', '30', 'London']
['Charlie', '22', 'New York']
```

In [55]:
```python
# Writing to a CSV file
import csv
data = [
    ["Name", "Age", "City"],
    ["Ali", 30, "New York"],
    ["Ahmed", 25, "London"]
]
with open("data.csv", "w", newline="") as csvfile:
    writer = csv.writer(csvfile)
    writer.writerows(data)
```

In [56]:
```python
# Reading CSV as dictionaries
# Using DictWriter and DictReader (Recommended)

import csv

# ---  Write to CSV using DictWriter ---
data = [
    {"Name": "Alice", "Age": 25, "City": "Paris"},
    {"Name": "Bob", "Age": 30, "City": "London"},
    {"Name": "Charlie", "Age": 22, "City": "New York"}
]

# Create and write CSV file
with open("data.csv", "w", newline="", encoding="utf-8") as csvfile:
    fieldnames = ["Name", "Age", "City"]
    writer = csv.DictWriter(csvfile, fieldnames=fieldnames)
    writer.writeheader()       # write column headers
    writer.writerows(data)     # write rows from list of dictionaries

print("✅ data.csv created successfully!\n")

# --- 2️⃣ Read CSV using DictReader ---
with open("data.csv", "r", encoding="utf-8") as csvfile:
    reader = csv.DictReader(csvfile)
    for row in reader:
        print(f"{row['Name']} is {row['Age']} years old and lives in {row['City']}.")
```

```
✅ data.csv created successfully!

Alice is 25 years old and lives in Paris.
Bob is 30 years old and lives in London.
Charlie is 22 years old and lives in New York.
```

7.3. JSON Files

Python provides the json module to handle JSON files.

In [57]:
```python
### Writing JSON to a file

import json

data = {
    "name": "Alice",
    "age": 30,
    "city": "New York"
}

with open("data.json", "w") as jsonfile:
    json.dump(data, jsonfile, indent=4)  # indent=4 for pretty printing
```

In [58]:
```python
## Reading JSON from a file
import json
```

```
with open("data.json", "r") as jsonfile:
    data = json.load(jsonfile)
    print(data)
    print(data["name"])
```

```
{'name': 'Alice', 'age': 30, 'city': 'New York'}
Alice
```

✅ Tips

- Always use with open(...) to automatically close the file.
- For CSV, use newline="" to avoid extra blank lines on Windows.
- For JSON, json.dump() writes to file and json.load() reads from file.

8. Modular Code: Functions, Scripts, and Modules

Modular programming is the practice of breaking a large program into smaller, reusable, and manageable pieces. In Python, modularity is achieved using functions, scripts, and modules. This approach improves readability, maintainability, and collaboration — especially in large projects.

8.1. Functions

A function is a reusable block of code designed to perform a specific task. It helps to avoid repetition and improve clarity.

In [59]:
```
# Example:
def greet(name):
    """Return a personalized greeting message."""
    return f"Hello, {name}!"

print(greet("Amel"))
```

```
Hello, Amel!
```

8.2. Scripts

A Python script is simply a file (.py) that contains code to be executed directly — often used for automation or standalone tasks.

In [60]:
```
# Example:

# File: hello.py

def greet(name):
    print(f"Hello, {name}!")

if __name__ == "__main__":
    greet("World")
```

```
Hello, World!
```

Then run it in the terminal:

python hello.py

or in jupeter :

%run hello.py

Note:

The condition

if **name** == "**main**":

ensures that certain code runs only when the script is executed directly, and not when it's imported as a module.

8.3. Modules

A module is any Python file (.py) that defines functions, classes, or variables — which can be imported and reused in other files.

In [61]:
```python
# Example:

# File: math_utils.py

def square(x):
    return x ** 2


# In another file:

# import math_utils

# print(math_utils.square(4))
```

Import variations:

from math_utils import square

print(square(5))

or

import math_utils as mu

print(mu.square(5))

8. 4. Organizing with Packages

A package is a **directory** that groups multiple **modules** together. It must contain an **init**.py file (even if empty).

Example structure:

```
project/
│
├── utils/
│   ├── __init__.py
│   ├── math_utils.py
│   └── string_utils.py
│
└── main.py
```

Then in main.py:

from utils.math_utils import square print(square(3))

Testing – each module can be tested independently.

💡 Best Practices

- Keep each module focused on one logical purpose.
- Use clear, descriptive names for functions and modules.
- Include docstrings ("""Description""") for documentation.
- Avoid circular imports (modules importing each other).

8.5. Libraries and Packages Installed with pip

A library (or package) in Python is a collection of pre-written modules that provide reusable functionality — such as data processing, visualization, machine learning, or web development. Instead of rewriting common features, you can install and import these libraries directly into your code.

In [62]:
```python
# Installing Libraries with pip: Basic commands:
# Install a library
# pip install library_name

# Example
!pip install numpy

# the use of a library
import numpy as np

array = np.array([1, 2, 3, 4])
print(array * 2)
# Output: [2 4 6 8]


# You can also import specific Built-in functions:

from math import sqrt
print(sqrt(16))  # Output: 4.0
```

```
Requirement already satisfied: numpy in c:\users\pc\anaconda3\lib\site-packages (2.1.3)
[2 4 6 8]
4.0
```

8.6. Using Built-In Modules

Python comes with a rich Standard Library, which provides many built-in modules ready to use — no installation required. These modules offer pre-written functions for common programming tasks such as mathematical operations, random number generation, date and time handling, file manipulation, and more.

In [63]:
```
# Example 1 – math Module

# Used for mathematical operations like square roots, trigonometric functions, logarithms, etc

import math

print(math.sqrt(16))      # Output: 4.0
print(math.pi)            # Output: 3.141592653589793
print(math.factorial(5))  # Output: 120
```

```
4.0
3.141592653589793
120
```

In [64]:
```
# Example 2 – random Module

# Used for generating random numbers or choosing random elements.

import random

print(random.randint(1, 10))               # Random integer between 1 and 10
print(random.choice(["a", "b", "c"]))      # Randomly selects one element
print(random.sample(range(10), 3))         # Selects 3 unique numbers
```

```
1
a
[9, 5, 2]
```

In [65]:
```
# Example 3 – datetime Module

# Used to work with dates and times.

import datetime

now = datetime.datetime.now()
print("Current date and time:", now)

today = datetime.date.today()
print("Today's date:", today)
```

```
Current date and time: 2025-10-12 21:46:44.094247
Today's date: 2025-10-12
```

In [66]:
```
# Example 4 – os Module

#Used for interacting with the operating system (creating folders, listing files, etc.).

import os

print(os.name)             # Shows the name of the operating system
print(os.getcwd())         # Shows the current working directory
print(os.listdir("."))     # Lists files in the current folder
```

```
nt
C:\Users\pc\Course_Boustil\Chapter3
['.ipynb_checkpoints', 'chapter3.ipynb', 'chapter3_part1.ipynb', 'data.csv', 'data.json', 'exa
mple.txt', 'SalesProject', 'TP3_Python_Programming_Essentials.ipynb']
```

✅ Tips : Writing Clean and Professional Python Code

- **Follow Naming Conventions**

Entity	Convention	Example
Variable	`lowercase_with_underscores`	`customer_age`
Function	`lowercase_with_underscores`	`calculate_mean()`
Class	`PascalCase`	`DataCleaner`
Constant	`ALL_CAPS`	`PI = 3.14`

- **Use Comments and Docstrings**

Use **docstrings** to describe what a function or class does.
Use **comments** sparingly — only when the code isn't self-explanatory.

In [67]:
```python
# Example Calculate average of a list
def avg(numbers):
    """Returns the average of a list of numbers."""
    return sum(numbers) / len(numbers)
```